

Jamaa Next Door

A PLATFORM TO UNIFY MUSLIMS



Designing a safer way for Muslims to find
congregational prayer nearby.

PROJECT DOCUMENTATION

ANDROID PROTOTYPE / GERMANY / 2026

Prepared by [Student name]

HOW THIS PORTFOLIO IS ORGANIZED

Contents

This document presents both the finished prototype and the path that produced it. It is written for a reader who has not previously encountered the project.

01	Project Overview and Motivation	3-4
02	Problem, Users, Goals and Scope	5-6
03	Concept Development and Decisions	7-8
04	Privacy, Security and Ethics	9-11
05	Visual Design Process	12-13
06	Technical Approach and Project Structure	14-15
07	AI-Assisted Workflow	16-18
08	Prototype and Final Product	19
09	Conclusion	20

DRAFT STATUS

The writing and structure are complete enough for review. Bracketed fields and reconstructed phone screens should be replaced with final personal details and device captures before submission.

Project Overview

Jamaa Next Door is a location-aware Android prototype that helps Muslims discover or organize a nearby congregational prayer when attending a mosque is not practical.

Find

Create

Join

Pray together

The central idea

A user can see upcoming Jama'ahs in the surrounding area, review the prayer and countdown, open a details sheet, and join. If no suitable gathering exists, the user can create one by selecting the prayer, time, location and a location photo, then reviewing and confirming it.

What makes it different

The product is deliberately short-lived and local. A Jama'ah is sorted by its scheduled time and ends automatically when that time arrives. The interface emphasizes what matters in the moment: which prayer, where approximately, how long remains, and whether the user is participating.

Design intention

The prototype is not intended to replace mosques. It supports the gaps between fixed mosque schedules and everyday life: work, classes, meetings, travel, unfamiliar neighborhoods and temporary accommodation.

PROTOTYPE OUTCOME

A standalone APK now installs directly on Android without Expo Go and connects to Supabase for authentication, profile data and Jama'ah publication.

Why I Chose This Topic

The project began with a personal experience rather than a technology choice.

A difficult first experience in Germany

When I first came to Germany, I lived for two months in an area where there was no mosque nearby. In my religious practice, praying in congregation five times a day is an important obligation. Being unable to do that bothered me throughout the time I lived there.

People were present, but connection was missing

There were many Muslims in the area, yet there was no simple way for us to find one another at prayer time. The problem was not necessarily the absence of people; it was the absence of awareness, coordination and trust. Seeing that lack of unity affected me and stayed in my mind.

Why Germany is relevant

Many places in Germany do not have easy access to a mosque or religious center. Structured work, classes and meetings can also make people miss the mosque's scheduled Jama'ah. Travelers and newcomers face the same uncertainty. Jamaa Next Door explores whether a small local gathering can be organized safely at a later suitable time.

I wanted to turn the feeling of being religiously isolated into a practical way for nearby people to find one another.

Problem and Users

The visible problem is missing congregational prayer. Underneath it are four connected barriers that the product must address together.

Access

A mosque may be too far away, closed, unknown to a newcomer or unreachable during a short break.

Timing

A person may miss the mosque's Jama'ah because of work, classes, meetings, travel or changing schedules.

Awareness

Other Muslims may be nearby, but nobody knows who is available or planning to pray.

Trust

Location sharing, unfamiliar hosts and identity checks create genuine safety and privacy concerns.

Primary user situations

The prototype is designed around workers, students, travelers, newcomers and local hosts. These are situations rather than rigid personas: the same person may be a host near home, a participant near work and a newcomer while traveling.

Core design question

How might an app make a nearby Jama'ah discoverable quickly without exposing exact private locations or asking users to trust strangers blindly?

Goals and Prototype Scope

The scope was kept focused on proving the central coordination loop rather than building a complete community platform.

What the prototype must prove

- A user can authenticate and complete a basic profile.
- Upcoming Jama'ahs can be discovered and ordered by time.
- A host can create, review and publish a Jama'ah.
- A participant can inspect details and join or leave.
- Sensitive location details can be revealed conditionally.
- The gathering disappears when its scheduled time arrives.

Deliberate boundaries

- The app does not replace mosque leadership or religious guidance.
- It does not claim that identity or selfie checks remove all risk.
- The current work is a prototype, not a fully staffed moderation service.
- Advanced community, donation and organization features are outside this document.
- Live operation still depends on correctly configured external services.

SUCCESS CRITERION

A first-time reader should understand the flow in one sentence: find a nearby prayer, review the details, join safely, or create one when nothing suitable exists.

From Note to Product Concept

The earliest notes focused on a simple question: if people do not already know the app exists, how can a host make a local prayer visible without creating noise or compromising privacy?

Features

* Want more people to join you? > Maybe not many people know about this app
 (show the number of users within a certain range. Like 1-2 km) >
 Donate, so that more people can know about this app.
 Your donation will only be used to advertise in your vicinity?



Early handwritten feature thinking. Source: Research/App Interface and Interaction.pdf.

Early concept

The first concept was close to a local broadcast: choose a prayer, choose a time and let nearby people know. That was useful as a starting point, but it did not yet explain who could see an address, how participation worked or when the event should end.

Concept development

The interaction became a sequence with explicit stages: create, review, confirm, publish, discover, open details, join and automatically end. Each stage was added because an ambiguous transition had created a real usability problem during testing.

Key Product Decisions

The product changed through a series of concrete decisions. Each decision links a user problem to an interface or system response.

Decision	Problem	Response
Direct APK	Expo Go added friction for testers.	Build a standalone Android package that installs directly.
Magic link	Passwords create extra setup and recovery work.	Use email authentication and return through an app deep link.
Review, then confirm	Publishing immediately can create mistakes.	Show a review state with Edit and Confirm Jama'ah actions.
Home-based details	A separate blank route broke context and crashed.	Open details in a bottom sheet over the time-sorted home list.
Approximate first	Exact addresses should not be public.	Show the area publicly and reveal exact details only when permitted.
Automatic ending	A Conclude button made the host responsible for cleanup.	End the Jama'ah at its scheduled start time and remove it from discovery.

DECISION RULE

When a result did not match the intended behavior, I described the observed behavior, the expected behavior and the parts that must remain unchanged before requesting another implementation.

Privacy Risk Landscape

Privacy is not an additional feature in Jamaa Next Door. The central interaction involves strangers, time, identity and physical location, so privacy decisions shape the product architecture.

Risk	Possible harm	Priority
Exact location	A private home or meeting place could be exposed to unrelated users.	High
Location photo	An image may reveal entrances, names, faces or identifying details.	High
Selfie verification	Biometric-like identity material creates storage, access and liability risks.	High
Messages	Moderation can feel like surveillance if readers and conditions are unclear.	Medium
Push notifications	Tokens and notification text can disclose participation or religious activity.	Medium
Abuse and reports	Bad actors may create misleading gatherings or target participants.	High

Privacy principles

Collect only what the core interaction needs; keep sensitive media private; enforce rules on the server rather than only in the interface; explain who can access information; and record privileged actions where accountability is required.

Safeguards in the Prototype

The prototype applies layered safeguards. No single measure is treated as sufficient on its own.

Area	Current safeguard
Location	Public discovery returns approximate information. Exact address and location media are restricted to the host or permitted active participants.
Storage	Selfie verification and Jama'ah location images use private Supabase Storage buckets with authenticated, ownership-based policies.
Database	Row Level Security is enabled on application tables. Application functions require authenticated users, while privileged jobs are restricted to trusted server roles.
Participation	Joining is separated from browsing. The server, not only the screen, decides whether a user may join and see protected details.
Accountability	Reports, blocks, notifications, attendance and deletion requests have dedicated data structures rather than being mixed into screen state.
Lifecycle	Cancelled and concluded Jama'ahs are excluded from discovery, and due gatherings are ended by a restricted scheduled operation.

RESIDUAL RISK

Technical controls reduce exposure, but they do not create complete trust. Clear consent, visible policy, moderation responsibility and real user testing are still necessary before public use.

Ethics and Transparency

Teacher feedback highlighted areas where transparency is as important as implementation. These questions are documented rather than hidden behind a polished interface.

Chat moderation

The product must state whether messages can be reviewed, who can review them, whose messages are accessible and under which reported circumstances. Moderation should create informed trust, not silent inspection.

Selfie verification

A professional third-party provider may be more trustworthy than building a verification operation internally. This would reduce direct handling of sensitive images, but provider terms, cost and data location still require scrutiny.

Age assurance

A checkbox confirming the user is 18 is only a declaration. It should not be presented as strong identity or age verification.

Language access

Arabic is an obvious language candidate. Right-to-left layout affects navigation, icons, alignment and mixed-language content, so it should be designed and tested deliberately rather than added as a text-only translation.

Transparency means explaining access before a sensitive action, not only after a problem occurs.

CURRENT POSITION

The prototype demonstrates privacy-aware architecture. It does not claim that moderation policy, identity verification or safeguarding operations are complete.

Visual Direction

The visual language aims to feel calm, local and trustworthy rather than corporate or overly technical. The interface uses contrast and time-based hierarchy to make the next action easy to understand.



Working design tokens used across the prototype and this portfolio.

Color

Deep teal gives the product a grounded identity; off-white reduces the clinical feeling of pure white; gold marks time and important action; sage supports quieter status and privacy information.

Type and shape

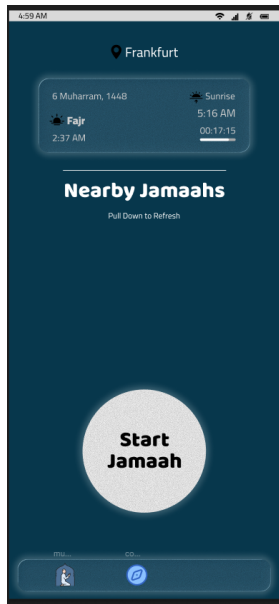
The documentation combines a human serif heading with a clear sans-serif body. In the app, rounded cards and bottom sheets create separation without turning every item into a generic dashboard tile.

Human over generic

The teacher's feedback challenged the project to move beyond an AI-generated appearance. The response is not simply more decoration. It is to use recognizable local language, real device captures, specific prayer information, carefully written consent and visual details derived from the actual use context.

Interface Development

An early Figma direction placed the current location, date, prayer, sunrise and countdown inside a translucent hero card. This established the time-sensitive character of the app.



Early home interface. Source: Research/Figma Home interface.png.

What the early design established

The prayer name and countdown were given immediate priority. The location label anchored the interface in the user's surroundings, while the 'Nearby' section suggested discovery below the fold.

What needed refinement

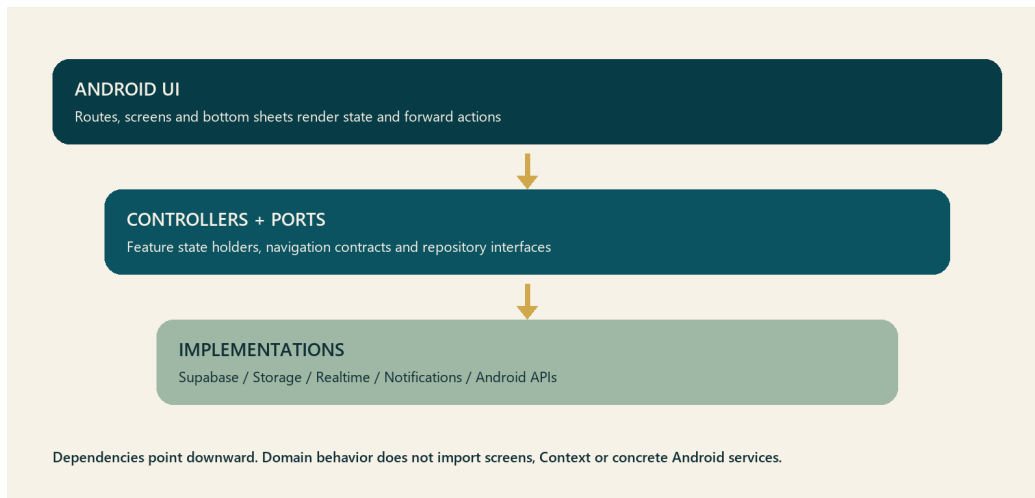
The first screen was visually attractive but still generic and incomplete. It did not yet explain participation, trust, privacy or the relationship between a prayer time and a user-created Jama'ah. Later iterations moved more responsibility into the home list and details bottom sheet.

Next visual pass before submission

Replace reconstructed screens in this draft with final device captures, check color contrast on a physical phone and ask a small group of Muslim users whether the tone feels familiar, respectful and trustworthy.

Technical Approach

The prototype uses a practical mobile-to-backend architecture. Screen code renders state and forwards actions; repositories and services own data and platform work.



One-directional dependency boundary used by the mobile prototype.

Mobile

Expo React Native and TypeScript provide the Android interface. Expo Router handles routes and deep links, while feature controllers and state holders keep submission and screen behavior outside route files.

Backend

Supabase provides Postgres, PostGIS, authentication, Row Level Security, Realtime and private Storage. Database functions enforce publication, participation and visibility rules.

Platform services

Expo notifications support push registration, Resend SMTP delivers authentication email, and EAS produces a signed standalone APK. These are explicit dependencies, not hidden assumptions.

Project Structure and Ownership

The mobile code is organized around responsibilities so that a change to one behavior does not require editing the entire application entry point.

Mobile owners

- app/: route composition and navigation entry points
- features/: screen-specific UI, controllers and state
- ports/: small interfaces for repositories and platform actions
- services/: Supabase, storage, notification and Android implementations
- providers/: explicit dependency assembly
- navigation/: navigation contracts and deep-link behavior
- theme/ and locales/: shared presentation configuration

Behavior boundaries

- UI renders state and forwards user intent.
- Controllers own transitions such as editing, reviewing and submitting.
- Repositories normalize database success and error results.
- Supabase functions remain the authority for protected data.
- Screen-local state owns the selected bottom sheet, avoiding unnecessary globals.
- The app entry point initializes and assembles dependencies only.

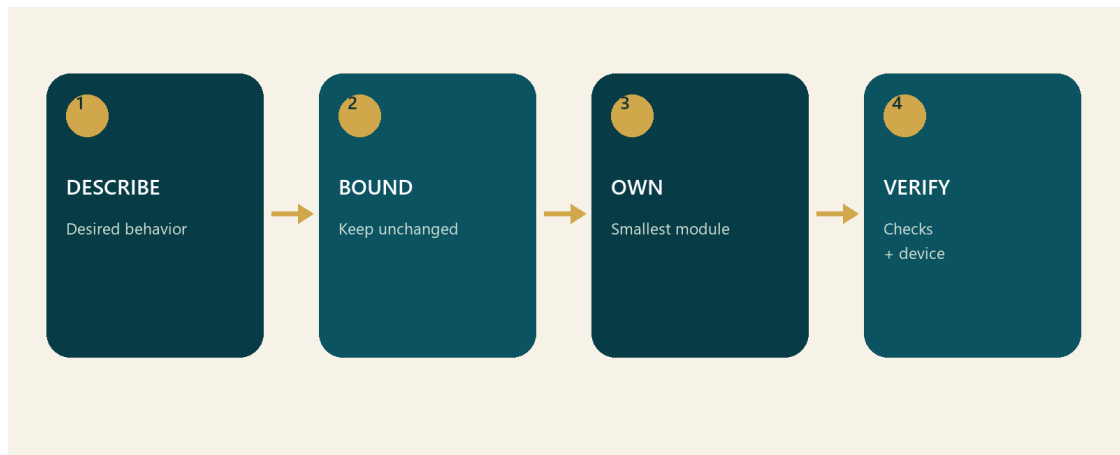
Android UI -> controller / state holder -> use case or repository interface
Repository implementation -> Supabase, Storage, Realtime, notifications and Android APIs

PRACTICAL ARCHITECTURE

Abstraction was introduced only where it reduced coupling or made behavior testable. The goal was not to reproduce a large enterprise architecture inside a prototype.

How the AI Workflow Changed

AI was used as an implementation collaborator, but the quality of the result depended on how I framed tasks, constrained changes and verified behavior.



Adapted from Research/safe-change-workflow.updated.pdf.

From broad request to controlled change

Early requests described the desired feature but not the ownership boundary or unchanged behavior. When results diverged from my vision, I began specifying the observed behavior, expected behavior, safety constraints, smallest owner module and verification steps.

The safe-change pattern

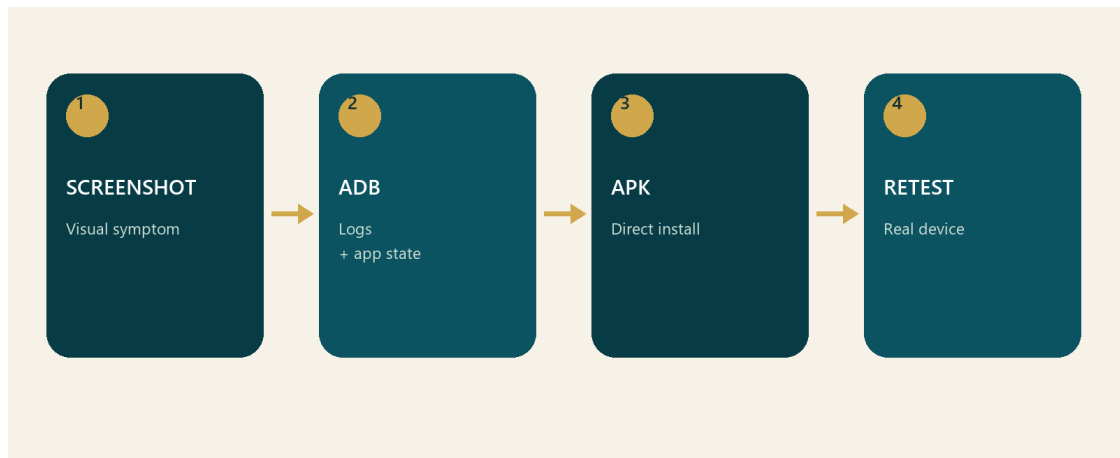
For each change I stated what should happen, what must remain unchanged and what obsolete behavior should disappear. I asked for explicit dependency injection rather than imports from the application entry point, avoided new global state and required syntax, import and server smoke checks.

Why this mattered

This turned prompting into an engineering control. It reduced unrelated edits and made failures easier to locate. The approach is documented in the research PDF as a reusable workflow for adding, changing, removing, fixing and refactoring behavior.

From Screenshots to Device-Level Testing

The testing workflow became progressively more direct as the prototype moved from Expo Go to a standalone APK.



Stage 1: screenshots and descriptions

At first, I tested on my phone and sent screenshots with written descriptions. This was useful for visual issues, but it could miss the exact tap sequence, application state and device logs.

Stage 2: USB debugging and ADB

I enabled developer options, USB debugging and the required install permissions. The phone could then be connected directly. The APK was installed over USB, the app was launched without Expo Go, and logs could be inspected while I repeated the issue.

Stage 3: observed interaction

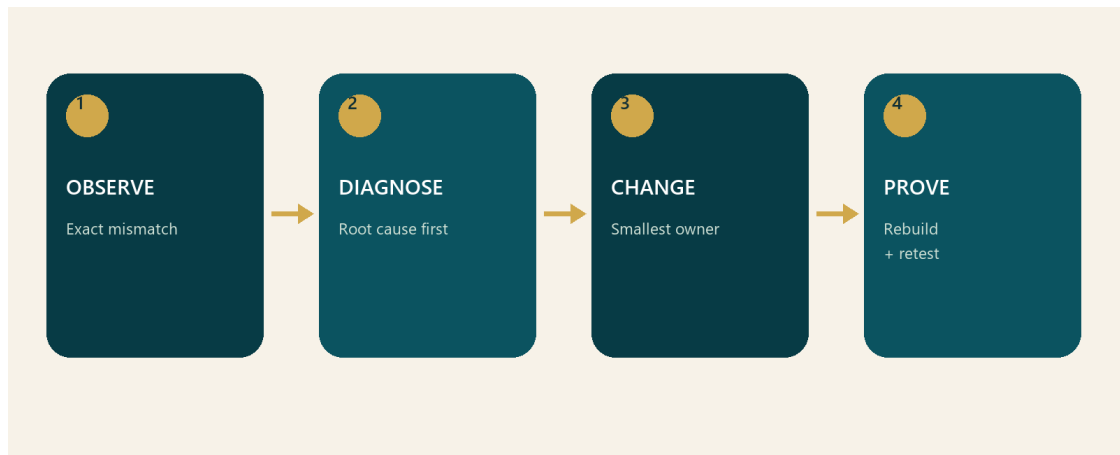
For one interaction, I recorded a similar app and asked the AI to observe the specific behavior I wanted. The task was not to copy the other product visually; it was to identify the interaction pattern, describe why it worked and adapt that behavior to Jamaa Next Door while preserving the app's own design and privacy rules.

EXAMPLE

The Jama'ah details experience changed from a separate route into a bottom sheet over the home list after the desired reference interaction and the existing app flow were compared directly.

Browser Control and Corrective Prompting

Some prototype failures were not inside the React Native code. Authentication depended on connected services, DNS and redirect configuration, so I allowed controlled browser access for configuration work.



Browser-assisted setup

With explicit permission, the AI could work through Namecheap DNS, Resend domain verification and Supabase authentication settings using my existing signed-in browser. I retained control over approvals and sensitive actions. Credentials were configured in their intended dashboards and were not copied into documentation.

Correcting mismatched results

When the app returned to the email screen after a magic link, I did not simply ask for another redesign. I described the exact reproduction sequence, asked for the root cause before changes, rebuilt the APK, cleared application data and repeated a cold-start deep-link test.

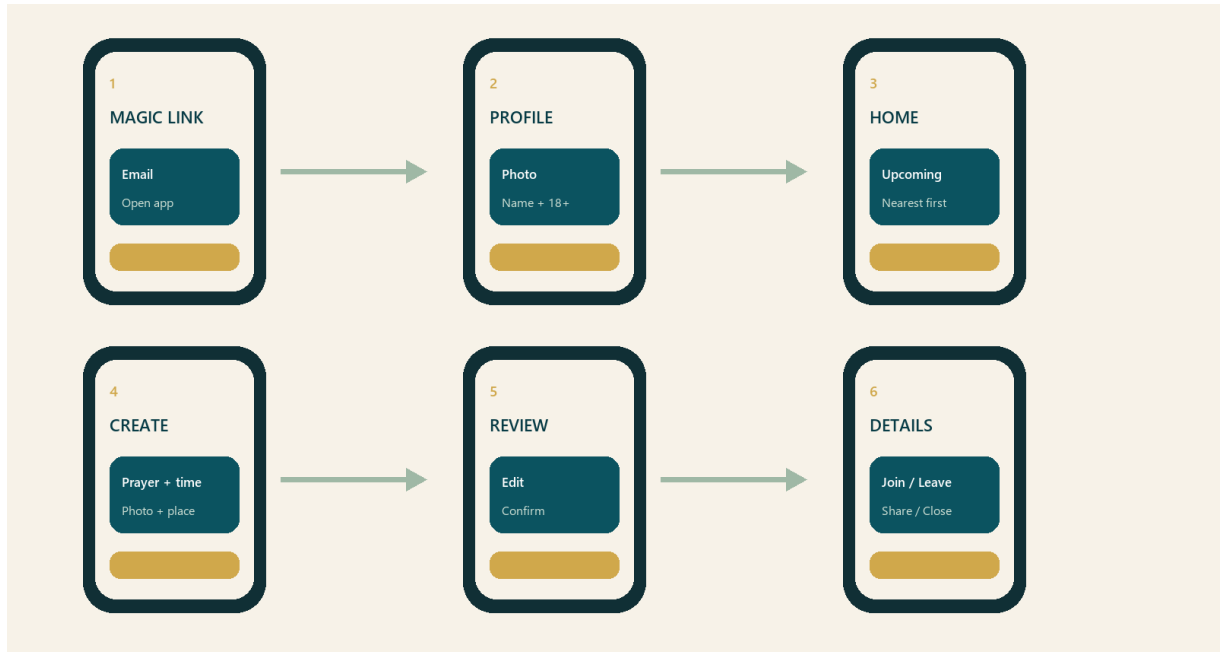
Examples of iterative corrections

The same method was used when profile setup had no Continue action, when Confirm Jama'ah raised a crypto compatibility error, when publication opened a blank page, and when the intended post-publication destination changed to the home list.

Observe -> state the mismatch -> preserve what works -> change the smallest owner -> rebuild
-> test on the real device.

Prototype and Final Product

The working prototype now demonstrates the complete central journey as a standalone Android application.



Draft reconstructions of the implemented journey. Replace with final device screenshots before submission.

Current flow

A user requests a magic link, returns to the app, completes a profile and reaches the Jama'ah home. The user can review upcoming gatherings or create one by selecting the prayer, scheduled time, exact location and required location photo. Confirmation uploads the image, creates the record and returns to a refreshed home list.

Participation

Tapping a home card opens a details bottom sheet. Depending on role and status, the sheet presents Join, Leave, Cancel, Share and Close actions. Shared text contains only approximate area information. Protected location details remain conditional.

Honest implementation status

The APK installs and launches without Expo Go, and the core Supabase-backed workflow has been implemented. This remains a prototype: public release would require final policy decisions, broader user testing, production monitoring and fully reviewed service configuration.

Conclusion

Jamaa Next Door began with a personal experience of religious isolation and developed into a working Android prototype with a defined interaction model, backend rules and a traceable design process.

What the project demonstrates

The project connects product design, mobile implementation and responsible system thinking. The main achievement is not only that a Jama'ah can be created. It is that creation, discovery, participation, protected location access and automatic ending are treated as one coherent lifecycle.

What changed in my process

I learned to make my intentions testable. Instead of treating AI output as a finished answer, I used explicit constraints, smaller module ownership, real-device observation and repeated verification. When the result did not match my vision, I improved the problem description and acceptance criteria rather than accepting a generic solution.

Final position

The prototype now communicates the core idea clearly: nearby Muslims should be able to find one another for prayer without exposing more information than the moment requires. The remaining responsibility is to present that idea honestly, validate it with the people it is intended for and ensure that trust is visible in both the interface and the system behind it.

Software is created for people. In this project, trust, language, timing and visual tone are part of the functionality.

BEFORE FINAL SUBMISSION

Replace bracketed metadata and draft reconstructions, proofread names and terminology, and confirm that every screenshot reflects the final installed APK.